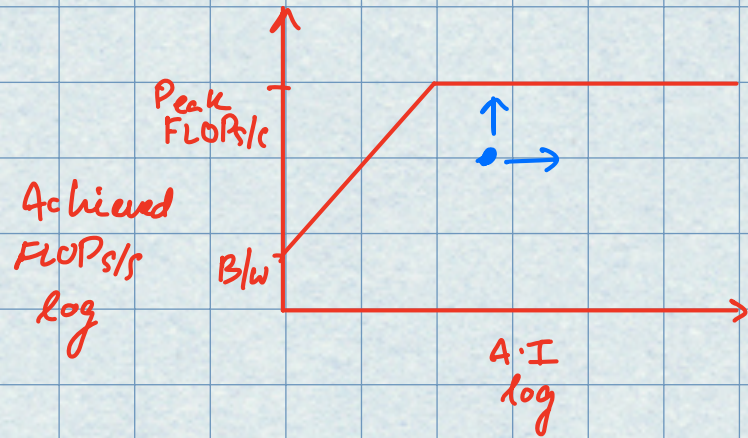


More clarity on how compute and memory fetching happens together.



To go $\rightarrow$ is hard

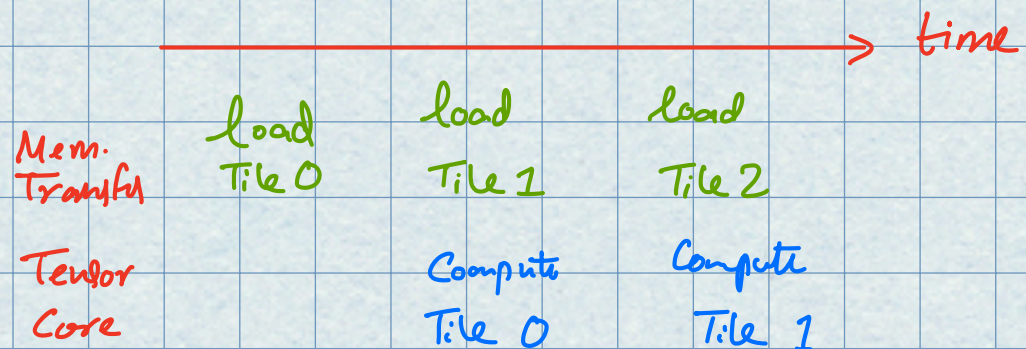
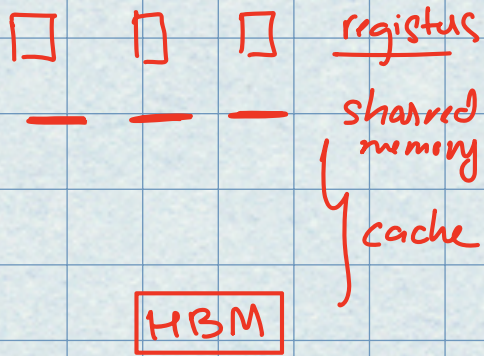
can go $\uparrow$ by clever memory access patterns.

Tiled Matrix Multiplication

$$\begin{bmatrix} \text{tile} \\ \square \end{bmatrix} \begin{bmatrix} \text{tile} \\ \square \end{bmatrix} = \begin{bmatrix} \text{tile} \\ \square \end{bmatrix}$$

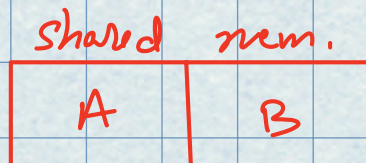
Want: $T_{\text{compute}} > T_{\text{comm}}$
 A.I. operation > Peak A.I. hardware

Mem. Hierarchy



Problem: when load tile 1 running, can override loaded tile 0 data.

Solⁿ: shared memory can be partitioned and used in ping-pong buffer configuration.



step 0: HBM tile 0 → Buffer A

step 1: HBM tile 1 → Buffer B
Tensor core compute tile 0

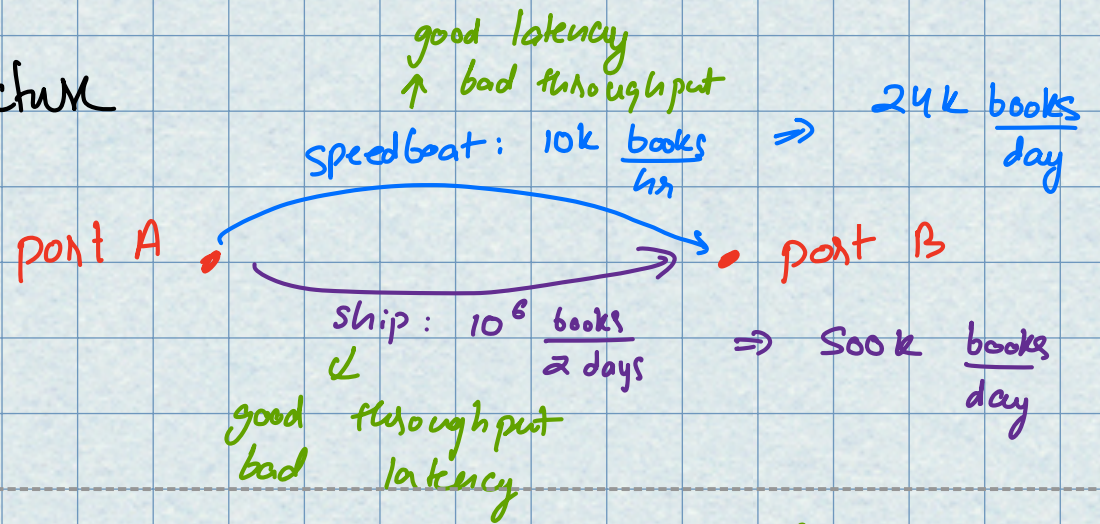
shared memory vs cache

↳ tensor cores and multiple threads use it to get data for compute at once.

↳ store data that MAYBE used next.

CPU vs GPU Architecture

Latency vs Throughput :

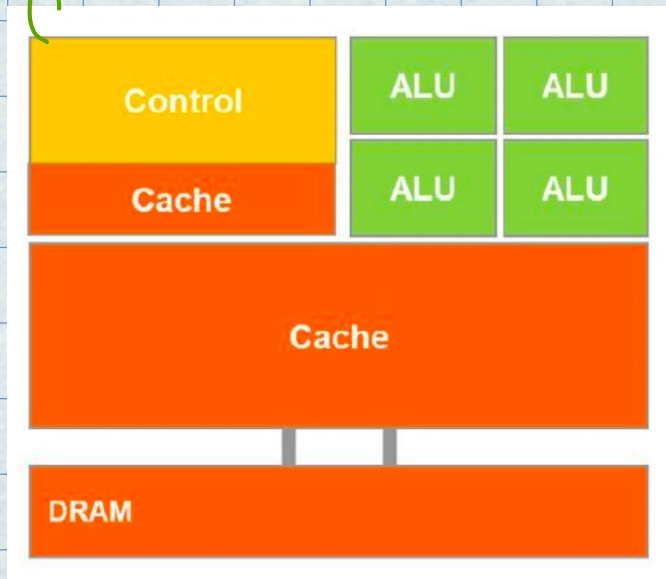


CPU

time b/w request sent and o/p received

- ↳ Latency-oriented design
- ↳ Sequential instruction flow

can do branch pred., pipeline, etc.



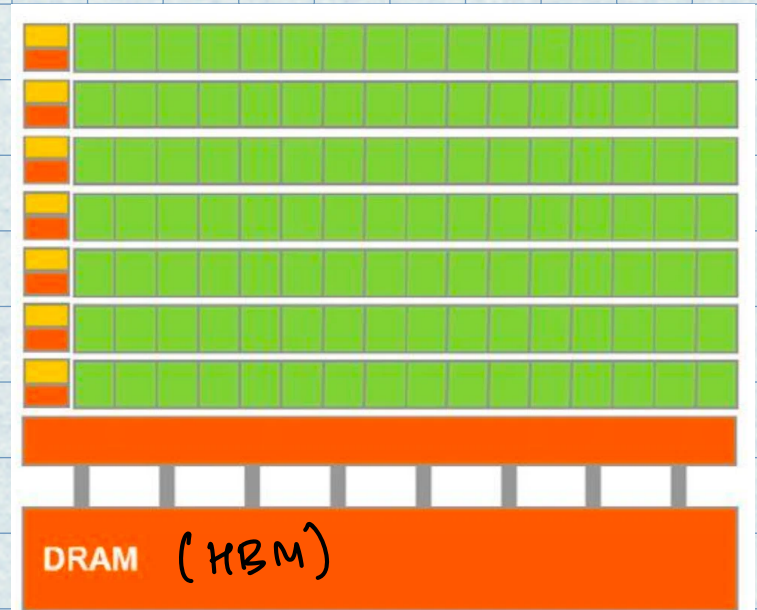
one core
↓
can be many

GPU

request fulfilled per unit time

- ↳ Throughput-oriented design.

multiple cores

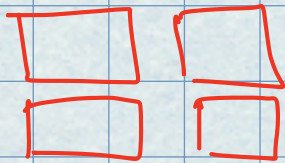


- bigger control & ALUs ^{→ individually more capable & faster}
- designed to make sequential control flow faster

for this reason

- Cache is bigger so cost to go to DRAM is not as big.

→ Say 4 cores



You can write parallel program but cores make one screen go fast.

- big bank of logic units
- small control, cache
- no optimisations like branch

predictions.

- Can afford to go to DRAM.
- Due to above, latency is higher

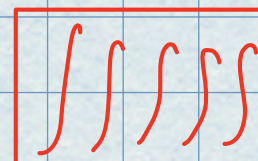
- Problem: having higher latency means our threads have to wait $\Rightarrow$ hurt throughput.

Solⁿ: Latency Hiding

#Threads $\ggg$ #Cores

CPU Threads vs. GPU ~~Threads~~ Warps

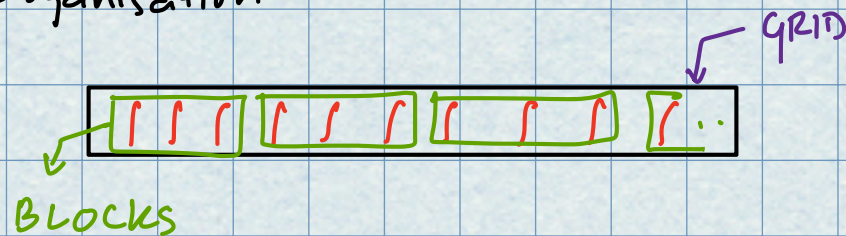
∫ CPU Thread



GPU warp
(group of 32 threads)

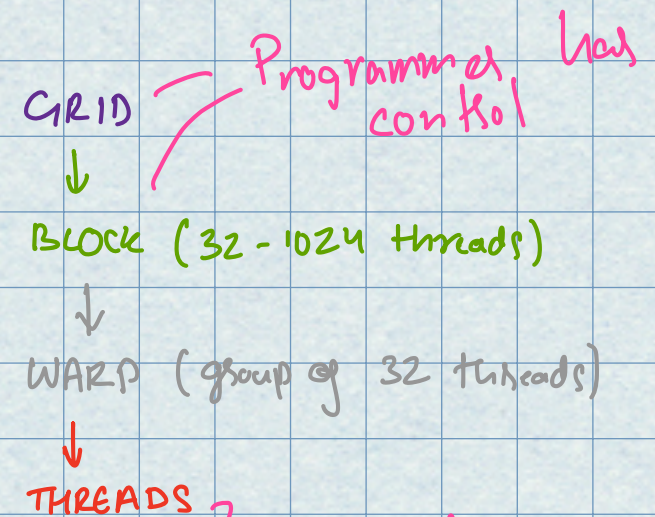
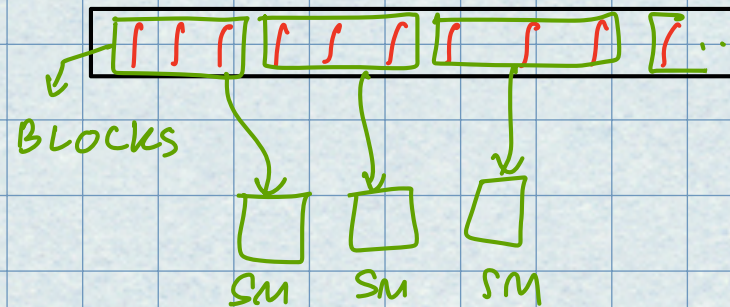
complexity	complex	Simple
life span	long	died after process
Scheduling overhead	different program counted per thread	same PC, threads operate on diff. data

Thread Organisation



Grid composed of blocks. All blocks have same no. of threads.

CUDA takes BLOCKS and puts them on SM.



→ Threads can cooperate with each other within a block

2 CUDA code s.t. same code runs on each thread.

Ex) RGB → Grayscale (pixel wise op.)

GRID → full image (1024×1024 px $\Rightarrow$ ~1 mil. thread for single pixel)
BLOCK → we define. say 256

$$\Rightarrow \text{no. of BLOCKS} = \frac{1024 \times 1024 \text{ threads}}{256} = \underline{\underline{4096}}$$

All independent operating units.

They sit in a queue and CUDA scheduler puts them in an empty SM.

Why not all threads in one block

SM has limited registers (~65k). Cannot run all threads.

Plus blocks give us independent, schedulable tasks.

Why not just one thread in a block

→ scheduling overhead

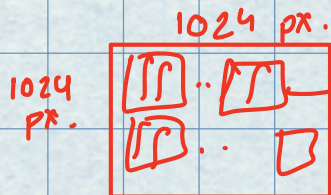
→ concept of WARPs which share PC (proceed in lock style)

↳ make 32 threads go in parallel.

∴ a block needs to have min. of 32 threads.

Can also arrange BLOCKS in 2D for easier programming.

ex)
image



→ 16x16 block (256 threads)

→ Multiple blocks can also be assigned to same SM.

↓
the SM can compute
on the blocks

no parallelly

← concurrently

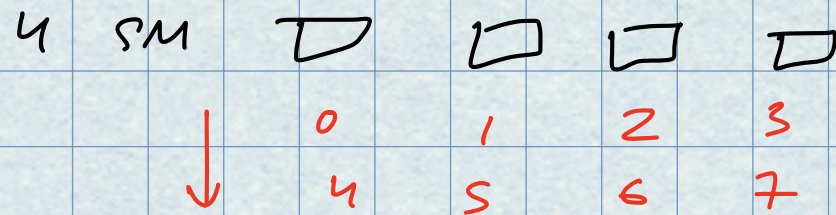
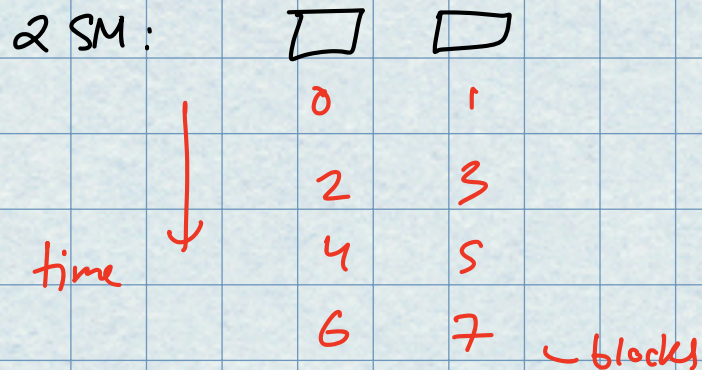
→ but still no cooperatⁿ
on those blocks

on one SM { [block 0] [block 1] }

if block 0 waiting for data,
block 1 threads will start to run.

→ GPU thread switching is extremely fast.

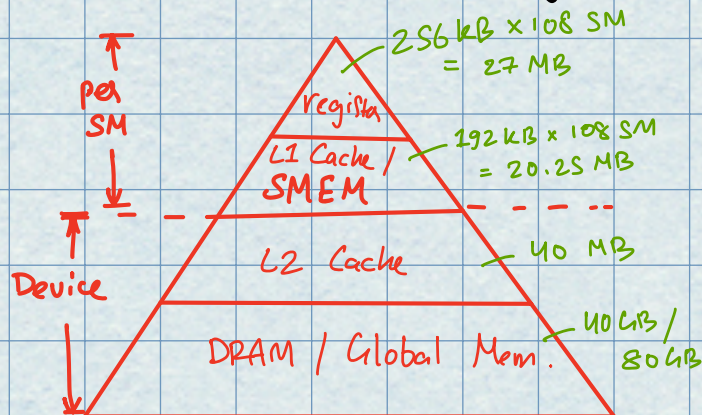
Ex) 8 blocks



if we have 7 or 9 blocks, we would get idle SMs.

⇒ we try to make no. of blocks some multiple of SMs.

GPU Memory Hierarchy



SMEM := shared memory.

No L3 Cache?

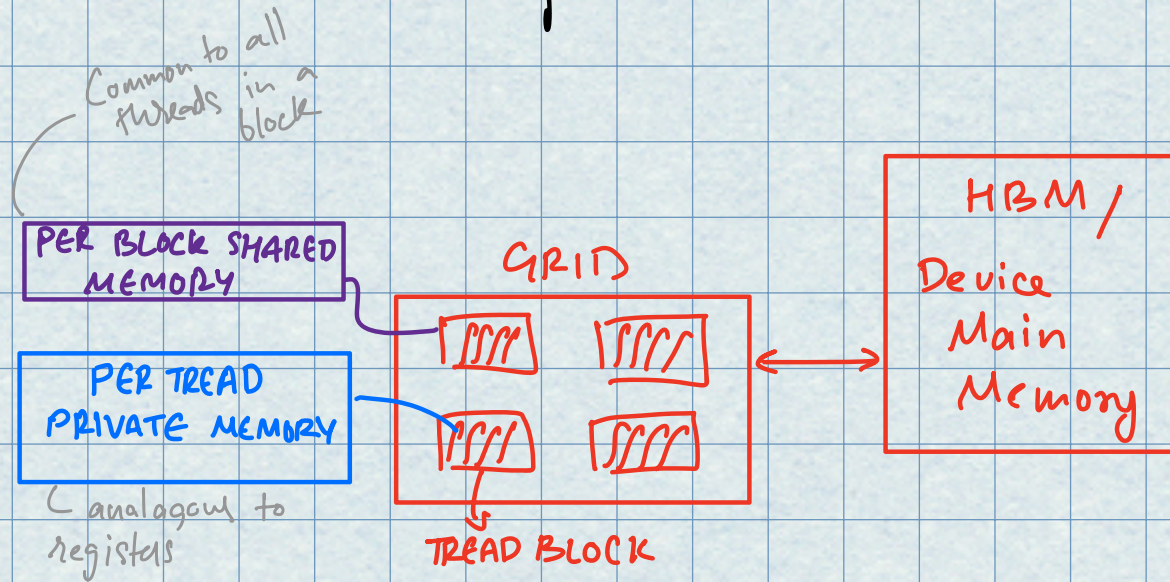
→ Cache is used to reduce memory latency

→ we do lots of latency hiding via warps.

SMEM

- ↳ we can declare and use this as we like
- ex) `--shared-- array[100];`
- ↳ Cache can't be used like this.

more on GPU Memory

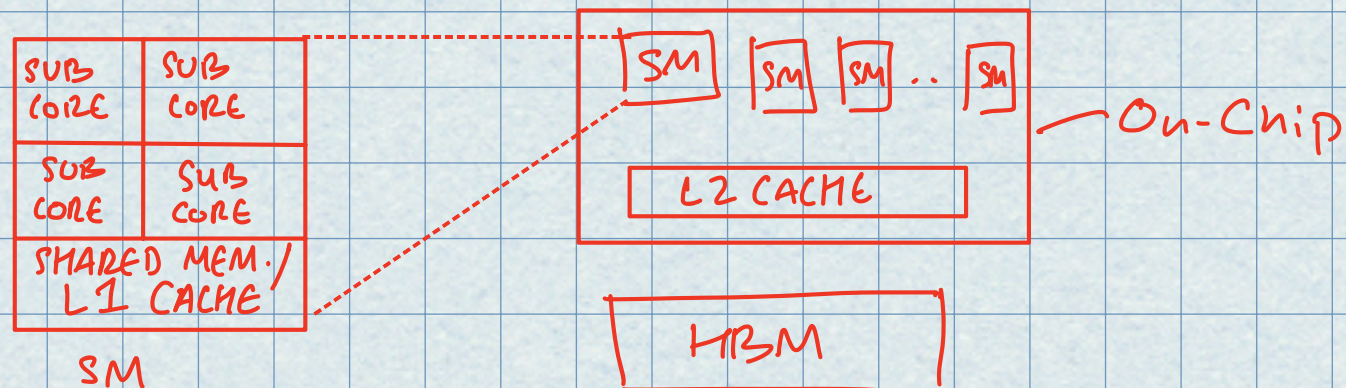


new tech

Distributed Shared Memory

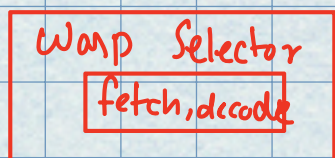
- ↳ between SMEM and L2 Cache
- ↳ In H100 (and later) we have a group of 18 SMs (called GPC) and they coordinate with distributed shared memory.
- ↳ Only present in H100 and later.

More on GPU architecture - Compute specific.



Sub Core :

- ↳ Can fetch - decode - execute
- ↳ Executes threads in warps (group of 32 threads running together).
- ↳ At a time, a sub-core can hold 4 warps and switch b/w them.



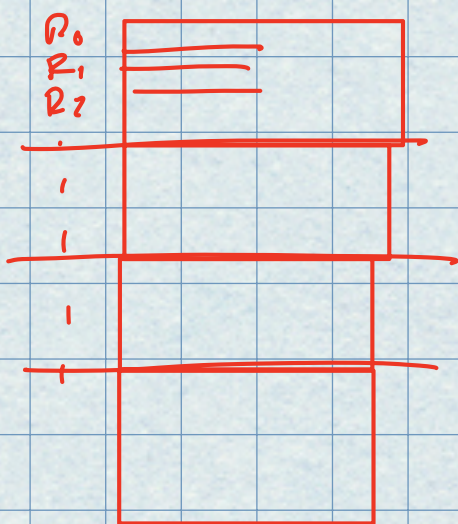
Warp :  operate in lock style

SIMD : single instⁿ multiple data

SIMD fp32 functⁿ unit
SIMD int
Tensor core
load store unit

control is shared across units.

Warp Selector
Fetch, decode



Registers for the 4 warps.

So many registers, very quick to switch warp.

On CPU, changing b/w threads means putting everything from registers to stack.

Nvidia A100 SM "sub-core"

 = SIMD fp32 functional unit, control shared across 16 units (16 x MUL-ADD per clock *)

 = SIMD int functional unit, control shared across 16 units (16 x MUL-ADD per clock *)

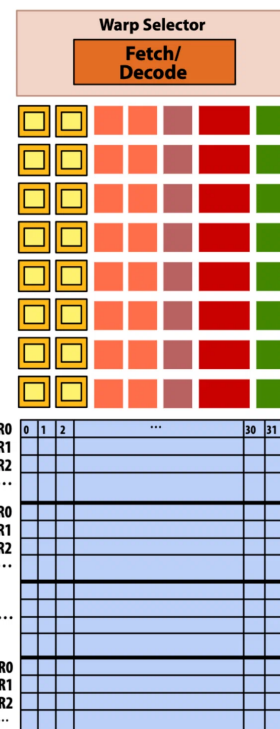
 = SIMD fp64 functional unit, control shared across 8 units (8 x MUL-ADD per clock **)

 = Tensor core unit

 = Load/store unit

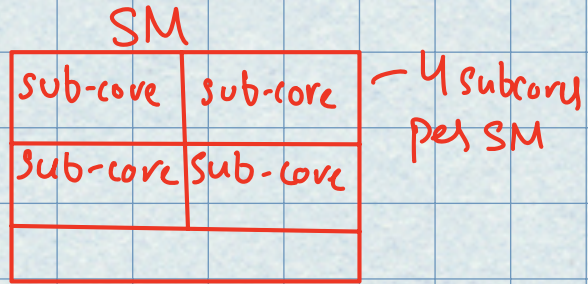
* one 32-wide SIMD operation every two clocks

** one 32-wide SIMD operation every four clocks



Ref. Diagram.

GPU Compute Architecture



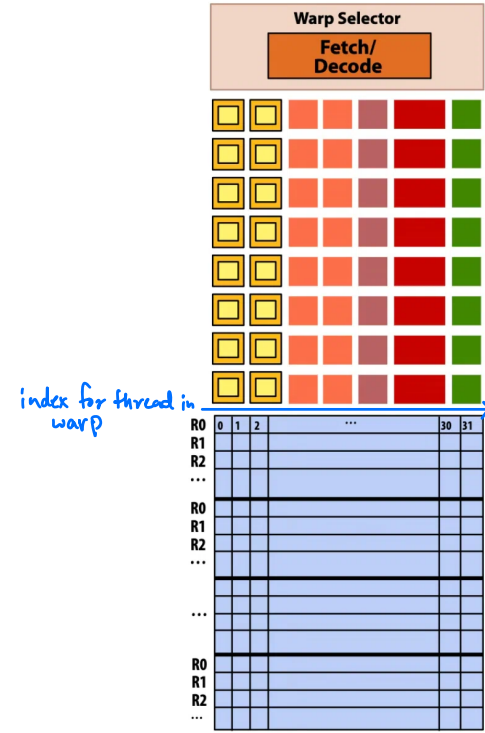
At a time, a Subcore
can work on one
warp

Nvidia A100 SM “sub-core”

-  = SIMD fp32 functional unit, control shared across 16 units (16 x MUL-ADD per clock *)
-  = SIMD int functional unit, control shared across 16 units (16 x MUL/ADD per clock *)
-  = SIMD fp64 functional unit, control shared across 8 units (8 x MUL/ADD per clock **)
-  = Tensor core unit
-  = Load/store unit

* one 32-wide SIMD operation every two clocks

**** one 32-wide SIMD operation every four clocks**



Capacity
for 4
warps
of 32 threads
each.

When we run a process we have a



Blocks are assigned to SMs. An SM can have multiple blocks assigned.

BLOCKS have WARPS which is a h/w concept. You don't interface with it in s/w.

Why have $4 \text{ warps} \times 32 \frac{\text{threads}}{\text{warp}} = 128 \text{ threads}$ rather than a warp of 128 threads?

→ For more fine grained parallelism and latency hiding.

← instruction fetch decode only needs to be done once for many threads. { SIMD architecture or vector processing. } → if one warp waiting for memory, we can switch warps and do some computation.

processor execution ↓

Threads in a warp move in lock style.

1. $\text{result}[i] = a[i] + b[i];$
2. $\text{if} (\text{result}[i] < 10) \quad \text{---} \textcircled{1}$
3. $\text{result}[i] *= 2;$
4. $\text{else} \quad \text{---} \textcircled{2}$
5. $\text{result}[i] *= 10;$

Say half my threads branch to $\textcircled{1}$ and other to $\textcircled{2}$.

↓

WARP DIVERGENCE

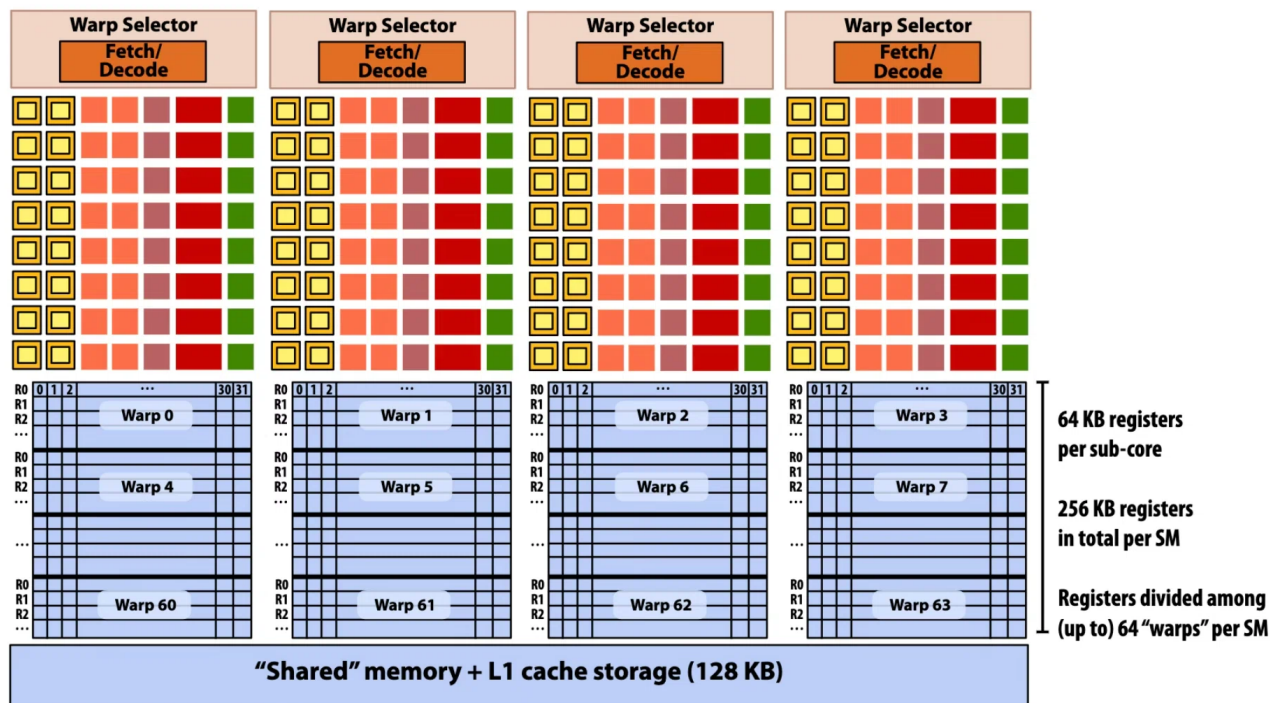
↳ Processor still goes step by step but threads not take path ① get masked and do not execute ①.

↳ Same for ②.

↳ Downside is inefficiency, idle compute.

#more on Parallelism

Single Nvidia A100 SM



In total 256 KB of registers -
Split among 4 subcores.

Say 1 register $\rightarrow$ 8 bytes

#registers $\rightarrow$ 32K

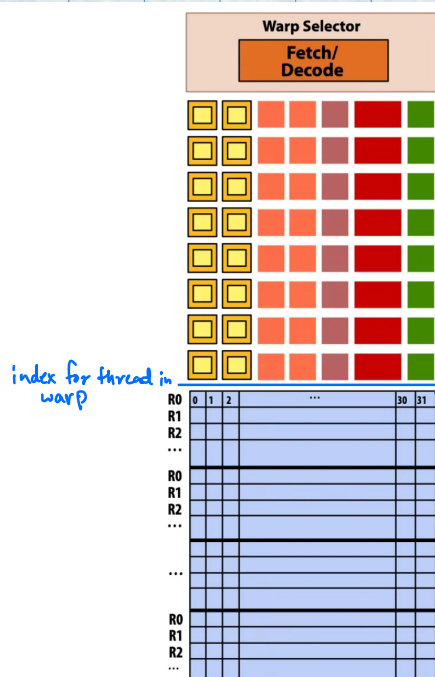
SO MANY REGISTERS !!!

Nvidia A100 SM "sub-core"

- = SIMD fp32 functional unit, control shared across 16 units (16 x MUL-ADD per clock *)
- = SIMD int functional unit, control shared across 16 units (16 x MUL/ADD per clock *)
- = SIMD fp64 functional unit, control shared across 8 units (8 x MUL/ADD per clock **)
- = Tensor core unit
- = Load/store unit

* one 32-wide SIMD operation every two clocks

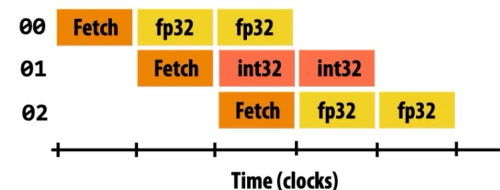
** one 32-wide SIMD operation every four clocks



fp32, int units are separate.
Can do those operations together.

Instruction stream for CUDA threads in a warp...
(note in this example all instructions are independent)

```
00 fp32 mul r0 r1 r2
01 int32 add r3 r4 r5
02 fp32 mul r6 r7 r8
...
```



From A100 Datasheet

FP32 : 19.5 TFLOPs/sec

how do we come to this no. ?

Nvidia A100 SM "sub-core"

□ = SIMD fp32 functional unit,
control shared across 16 units
(16 x MUL-ADD per clock *)

■ = SIMD int functional unit,
control shared across 16 units
(16 x MUL-ADD per clock *)

■ = SIMD fp64 functional unit,
control shared across 8 units
(8 x MUL-ADD per clock **)

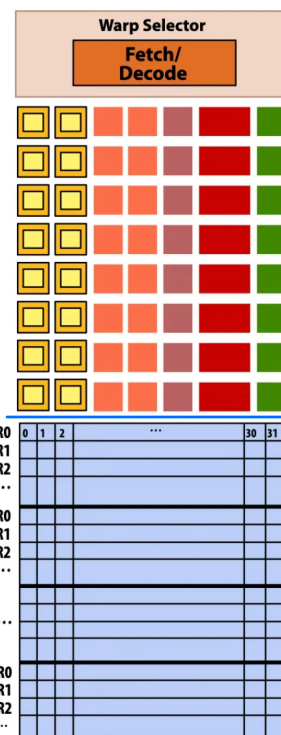
■ = Tensor core unit

Take care of -
main mem.
LOAD, STR
■ = Load/store unit

* one 32-wide SIMD operation every two clocks

** one 32-wide SIMD operation every four clocks

index for thread in
warp



1. How many FP32 functⁿ units?

A100 : 108 SM

4 subcore / SM

16 FP32 functⁿ units / subcore

$$\begin{aligned}\# \text{FP32 Funct}^n \text{ units} &= 108 \times 4 \times 16 \\ &= 6912\end{aligned}$$

2. How many FLOPs on 1 FP32 unit?

New Fact : 1 FMA instructⁿ / clock cycle
(datasheet)

faster to do $\left\{ \begin{array}{l} \text{matrix mul} \\ \text{from this} \end{array} \right.$

↳ Fused multiply add

$$C = a \times b + d$$

$$1 \text{ FMA} : C = a \times b + d$$

$$\Rightarrow 1 \text{ FMA} = 2 \text{ FLOPs}$$

How does FMA only take 1 clock cycle?

↳ This is a steady state number.

Each FMA may take say 6 clock cycles

The design is throughput orientated & units are pipelined.

When one FMA gets executed, the next one is already in pipeline stages.

3. How many FLOPs per sec? $1.41 \times 10^9 \text{ Hz}$
(1.41 GHz)

$$\begin{aligned}
 4. \text{ Total } & 108 \text{ SM} \times 4 \text{ Subcore} \times 16 \text{ FP32 units} \times 2 \frac{\text{FLOPs}}{\text{clock cycle}} \times 1.41 \times 10^9 \frac{\text{cycles}}{\text{s}} \\
 & = 19491 \times 10^9 \frac{\text{FLOPs}}{\text{s}} \\
 & \approx 19.5 \text{ TFLOPs/s}
 \end{aligned}$$

ex) How many clockcycles on FP32 unit if all threads in a warp want to do FMA?

32 threads /warp

16 FP32 unit

each unit does FMA in 1 clock cycle

⇒ 2 clock cycles.

Why not have 16 threads in a warp?

h/w Designers decision. Tradeoff of 2 times fetch/decode vs running instructⁿ twice.

Why not 32 FP32 units?

chip space on die.

